Sheaf-Cohomological Program Analysis: Unifying Bug Finding, Equivalence, and Verification via Čech Cohomology

HALLEY YOUNG, Microsoft Research, USA

We present a framework in which program analysis—type checking, bug finding, and equivalence verification—is organized as computing the Čech cohomology of a *semantic presheaf* over a program's *site category*. The presheaf assigns refinement-type information to observation sites (argument boundaries, branch guards, call results, error sites) and restricts it along data-flow morphisms. The zeroth cohomology group $\check{H}^0$ is the space of globally consistent typings. The first cohomology group $\check{H}^1$ classifies *gluing obstructions*—bugs, type errors, and equivalence failures—each localized to a specific pair of disagreeing sites.

This formulation yields three concrete results unavailable in prior work: (1) rk $\check{H}^1$ over $\mathbb{F}_2$ equals the minimum number of independent fixes needed to resolve all errors; (2) $\check{H}^1(\mathcal{U}, \mathcal{I}so) = 0$ is a *sound and complete* criterion (relative to the cover) for behavioral equivalence of two programs; (3) the Mayer-Vietoris exact sequence provides compositional obstruction counting, enabling incremental analysis. We prove soundness, completeness relative to the cover, and fixed-point convergence.

We implement the framework in Deppy, a Python analysis tool, and evaluate it on a suite of 375 benchmarks: 133 bug-detection programs, 134 equivalence pairs, and 108 specification-satisfaction checks. Deppy achieves **100% bug-detection recall** (69% precision, F1 = 81%), 99% equivalence accuracy with **zero false equivalences**, and 98% spec accuracy with **zero false satisfactions**—outperforming mypy and pyright, which report zero findings on unannotated code. The analysis models Python semantics as algebraic geometry: variables live on the generic fiber (non-None) unless on the closed nullable subscheme, integers form Spec($\mathbb{Z}$) with no bounded section (no overflow), and short-circuit evaluation defines an open-set topology on the presheaf. Key metatheorems are mechanized in 1 259 lines of Lean 4.

Additional Key Words and Phrases: sheaf theory, Čech cohomology, program analysis, refinement types, bug finding, equivalence checking

1 Introduction

Every program analysis collects *local facts* about individual program points and composes them into *global conclusions* about the whole program. Abstract interpretation does this via lattice fixpoints and widening [**?**]. Type inference propagates constraints through unification [**?**]. Hoare logic chains pre/post-conditions through weakest-precondition calculi [**?**]. Model checking explores reachable states [**?**].

These approaches share a deep structure: they all perform *local-to-global* reasoning. Sheaf theory, developed in algebraic geometry and algebraic topology, is precisely the mathematical theory of local-to-global composition. A *sheaf* assigns data to open sets of a space and specifies how local data *glue* into global data. When local data fail to glue, the obstruction lives in the first *Čech cohomology group* $\check{H}^1$.

In this paper, we show that program analysis—type checking, bug finding, and equivalence verification—can be unified as computing the Čech cohomology of a *semantic presheaf* over the program's *site category*. This is not a metaphor: the presheaf is a concrete data structure (implemented as a dictionary from sites to refinement-type sections), the coboundary operators are concrete algorithms (checking agreement on overlaps via SMT), and the cohomology groups are concrete invariants with direct engineering consequences.

Contributions.

Author's Contact Information: Halley Young, Microsoft Research, Redmond, WA, USA, halleyyoung@microsoft.com.

(1) **Framework** (§??). We formalize program analysis as sheaf cohomology over a site category with five site families, a semantic presheaf valued in complete lattices, and a Grothendieck topology satisfying identity, stability, and transitivity.
(2) **Three applications** (§??). (a) Bugs are gluing obstructions: $\check{H}^1 \neq 0$ iff the program has an error, and rk $\check{H}^1$ over $\mathbb{F}_2$ equals the minimum number of independent fixes. (b) Equivalence: $\check{H}^1(\mathcal{U}, \mathcal{I}so) = 0$ iff local equivalences glue to a global equivalence (sound *and* complete for the cover). (c) Specification verification: the product-cover construction factors VCs, with $\check{H}^1$-based elimination of redundant obligations.
(3) **Metatheory** (§??). Soundness (Theorem ??), completeness relative to the cover (Theorem ??), the Mayer-Vietoris exact sequence for programs (Theorem ??), and fixed-point convergence (Theorem ??). Key results are mechanized in Lean 4 with Mathlib.
(4) **Algorithms** (§??). Cover construction from Python ASTs, local solving via theory-family dispatch, bidirectional fixed-point synthesis (backward safe-input synthesis + forward output propagation), and interprocedural section transport via the Grothendieck transitivity axiom.
(5) **Implementation and evaluation** (§??). Deppy: full Čech pipeline covering 28 bug classes. Evaluated on 375 benchmarks (133 bug programs, 134 equivalence pairs, 108 spec checks): 100% bug recall, 99% equiv accuracy (zero false equivalences), 98% spec accuracy (zero false satisfactions), 77% H^1 rank accuracy. Key theorems mechanized in 1 259 lines of Lean 4.

2 Overview by Example

We illustrate the sheaf-cohomological approach on a small but instructive Python program with a subtle inter-procedural bug.

```
def normalize(values):
    total = sum(values)     # site U₁: total = sum(values)
    return [v / total        # site U₂: ERROR if total == 0
            for v in values]

def process(data):
    filtered = [x for x in data if x >= 0] # site U₃
    return normalize(filtered) # site U₄: call
```

Sites. The analysis identifies four observation sites: U_1 (argument boundary of `normalize`), U_2 (error site: division by `total`), U_3 (branch guard: `x >= 0` filter), U_4 (call result: `normalize(filtered)`).

Sections. At each site, the analysis assigns a *local section*—a refinement predicate over the variables in scope:

$$\sigma_1 : \mathit{values} : \texttt{list[int]}$$
$$\sigma_2 : \mathit{total} : \texttt{int},\ \mathit{viability}\text{: } \mathit{total} \neq 0$$
$$\sigma_3 : \mathit{filtered} : \texttt{list[int]},\ \forall x.\, x \geq 0$$
$$\sigma_4 : \mathit{filtered} : \texttt{list[int]},\ \forall x.\, x \geq 0$$

Overlap and obstruction. The restriction map from U_4 to U_1 (the call boundary) maps `filtered` to `values`. The section σ_4 restricted to U_1 says `values` is a list of non-negative integers—but does *not* guarantee `values` is non-empty. At site U_2, the viability predicate requires $\mathit{total} \neq 0$. If `values = []`, then $\mathit{total} = 0$, violating viability.

The sections σ_1 and σ_2 *disagree* on the overlap $U_1 \cap U_2$: σ_1 admits `values = []` but σ_2 requires $\sum$ *values* $\neq 0$. This disagreement is a 1-cocycle in $\check{C}^1$, and since it is not a coboundary (no local adjustment resolves it), it represents a nonzero class in $\check{H}^1$.

What standard tools report. `mypy`: no error (types match). `pyright`: no error (no type annotation issue). `Semgrep`: no match (no pattern for this bug class).

What Deppy *reports.*

$\mathbf{H^1}$ obstruction at overlap (U_1, U_2): `values` may be empty list, making `total = 0`.
Fix: guard `normalize` call with `if filtered:` (1 independent fix; rk $\check{H}^1 = 1$).

3 Formal Framework

We formalize program analysis as computing the Čech cohomology of a semantic presheaf. All definitions in this section are mechanized in Lean 4 (see supplementary material).

3.1 Program Site Category

Definition 3.1 (Site kind). A *site kind* is one of five families:
(1) ArgBoundary: function argument sites.
(2) BranchGuard: conditional/assertion sites.
(3) CallResult: inter-procedural call sites.
(4) OutBoundary: function output / return sites.
(5) ErrorSite: sites where partial operations may fail.

Definition 3.2 (Program site category). Given a program P, the *program site category* S_P is the category where:

- **Objects** are program sites $s = (\kappa, n)$ where κ is a site kind and n is a unique identifier (function name + line number).
- **Morphisms** $f : s \to t$ are *data-flow edges*: f exists iff information at s is available at t. Each morphism carries a *projection* $\pi_f : \mathrm{Keys}(t) \to \mathrm{Keys}(s)$ mapping refinement keys at t to those at s.
- **Composition**: $(g \circ f).\pi = f.\pi \circ g.\pi$ (contravariantly, as for a presheaf).
- **Identity**: $\mathrm{id}_s.\pi = \mathrm{id}$.

3.2 Semantic Presheaf

Definition 3.3 (Refinement lattice). A *refinement lattice* $(\mathcal{R}, \sqsubseteq, \sqcap, \sqcup, \top, \bot)$ is a complete lattice where:

- $\top$ = "no information" (any value is possible).
- $\bot$ = "contradiction" (no value is possible).
- $\sqcap$ = conjunction of refinement predicates.
- $\sqcup$ = disjunction of refinement predicates.
- The *information order*: $\phi \sqsubseteq \psi$ iff every state satisfying ψ also satisfies ϕ (i.e., ψ is more informative).

Definition 3.4 (Local section). A *local section* at site s is a pair $\sigma = (\tau, \phi)$ where τ is a carrier type (e.g., `int`, `list[T]`) and $\phi \in \mathcal{R}$ is a refinement predicate. A section *abstracts* a set of concrete states: $\gamma(\sigma) \coloneqq \{v : \tau \mid \phi(v)\}$.

Definition 3.5 (Semantic presheaf). The *semantic presheaf* is a contravariant functor

$$\mathcal{S}em : \mathrm{S}_P^{\mathrm{op}} \longrightarrow \mathbf{Lat}$$

where **Lat** is the category of complete lattices and monotone maps.

- For each site s, $\mathcal{S}em(s)$ is the lattice of local sections at s, ordered by information content.
- For each morphism $f : s \to t$, the *restriction map* $\mathcal{S}em(f) : \mathcal{S}em(t) \to \mathcal{S}em(s)$ restricts a section at t to what is observable at s. Concretely: $\mathcal{S}em(f)(\tau, \phi) = (\tau|_s, \phi \circ \pi_f)$ where π_f is the projection.

Functoriality: $\mathcal{S}em(\mathrm{id}_s) = \mathrm{id}$ and $\mathcal{S}em(g \circ f) = \mathcal{S}em(f) \circ \mathcal{S}em(g)$.

3.3 Grothendieck Topology

Definition 3.6 (Covering family). A *covering family* for site s is a finite set of morphisms $\mathcal{U} = \{u_i \xrightarrow{f_i} s\}_{i \in I}$ satisfying the *local determination property*: the natural map

$$\mathcal{S}em(s) \longrightarrow \prod_{i \in I} \mathcal{S}em(u_i)$$

is injective (separation) and, when composed with the equalizer of the double restriction $\prod_i \mathcal{S}em(u_i) \rightrightarrows \prod_{i<j} \mathcal{S}em(u_i \times_s u_j)$, is surjective (gluing).

Definition 3.7 (Grothendieck topology on S_P). The collection of covering families defines a Grothendieck topology J on S_P satisfying:

(1) **Identity.** $\{\mathrm{id}_s : s \to s\}$ is a covering family for s.
(2) **Stability.** If $\mathcal{U} = \{u_i \to s\}$ is covering and $t \to s$ is any morphism, then the pullback family $\{u_i \times_s t \to t\}$ is covering for t.
(3) **Transitivity.** If $\{u_i \to s\}$ is covering and each $\{v_{ij} \to u_i\}$ is covering, then $\{v_{ij} \to s\}$ is covering.

Remark 1. In the program setting: (1) Identity holds trivially. (2) Stability corresponds to *specialization*: if a cover works for site s, restricting it to a sub-site t (e.g., a branch of a conditional) still works. (3) Transitivity corresponds to *interprocedural composition*: if a function f is covered by its call sites, and each call site is covered by its arguments, then f is covered by the transitive closure. This is exactly the Grothendieck axiom that justifies summary-based interprocedural analysis (cf. IFDS/IDE [?]).

3.4 Čech Cohomology

Given a covering family $\mathcal{U} = \{u_i \to s\}$ and presheaf $\mathcal{S}em$, the Čech complex is:

$$C^0(\mathcal{U}, \mathcal{S}em) \xrightarrow{\delta^0} C^1(\mathcal{U}, \mathcal{S}em) \xrightarrow{\delta^1} C^2(\mathcal{U}, \mathcal{S}em)$$

Definition 3.8 (Čech cochains).

$$C^0(\mathcal{U}, \mathcal{S}em) \coloneqq \prod_i \mathcal{S}em(u_i) \qquad \text{(one section per site)}$$

$$C^1(\mathcal{U}, \mathcal{S}em) \coloneqq \prod_{i<j} \mathcal{S}em(u_i \times_s u_j) \qquad \text{(one section per overlap)}$$

$$C^2(\mathcal{U}, \mathcal{S}em) \coloneqq \prod_{i<j<k} \mathcal{S}em(u_i \times_s u_j \times_s u_k) \qquad \text{(one section per triple)}$$

Definition 3.9 (Coboundary operators).

$$(\delta^0 \sigma)_{ij} \coloneqq \mathcal{S}em(\mathrm{pr}_2)(\sigma_j) - \mathcal{S}em(\mathrm{pr}_1)(\sigma_i) \qquad \text{(disagreement on overlap)}$$

$$(\delta^1 \tau)_{ijk} \coloneqq \tau_{jk} - \tau_{ik} + \tau_{ij} \qquad \text{(cocycle condition on triples)}$$

where subtraction is in the lattice (complement of meet).

Definition 3.10 (Čech cohomology groups).

$$\check{H}^0(\mathcal{U}, \mathcal{S}em) := \ker \delta^0 = \{\sigma \in C^0 \mid \text{all overlaps agree}\}$$
$$\check{H}^1(\mathcal{U}, \mathcal{S}em) := \ker \delta^1 / \operatorname{im} \delta^0$$

The key interpretations:

- $\check{H}^0$ = the set of *globally consistent type assignments* (compatible families that glue).
- $\check{H}^1$ = the set of *gluing obstructions* modulo those removable by local adjustment (coboundaries).
- $\check{H}^1 = 0$ iff the sheaf condition holds for the cover: every compatible family glues uniquely.

3.5 The Sheaf Condition

Definition 3.11 (Sheaf condition). The presheaf $\mathcal{S}em$ is a *sheaf* with respect to topology J if, for every covering family $\mathcal{U}$ and every compatible family $(\sigma_i)_{i \in I}$ (i.e., $\delta^0 \sigma = 0$), there exists a unique $\sigma \in \mathcal{S}em(s)$ such that $\mathcal{S}em(f_i)(\sigma) = \sigma_i$ for all i.

Proposition 1. *$\mathcal{S}em$ satisfies the sheaf condition for cover $\mathcal{U}$ if and only if $\check{H}^1(\mathcal{U}, \mathcal{S}em) = 0$.*

Proof. ($\Rightarrow$): If $\mathcal{S}em$ is a sheaf, every cocycle $\tau \in \ker \delta^1$ arises from a compatible family (by definition of the Čech complex), which glues to a global section. Adjusting the local sections to the glued section shows $\tau \in \operatorname{im} \delta^0$.

($\Leftarrow$): If $\check{H}^1 = 0$, every 1-cocycle is a 1-coboundary. Given a compatible family (σ_i), the disagreements on overlaps form a 1-cocycle. Since it is a coboundary, there exist adjusted sections σ_i' with zero disagreement, which by the separation property of the cover determine a unique global section. (Formalized in Lean: `DeppyProofs.Presheaf.h0_lifts_to_section`.) □

4 Analysis Algorithms

The analysis proceeds in six stages, corresponding to the pipeline Parse → Harvest → Cover → Solve → Synthesize → Render.

4.1 Cover Construction

Given a parsed Python module, the Cover stage constructs the site category $\mathbf{S}_P$ and a covering family:

(1) Create ArgBoundary sites for each function parameter.
(2) Create BranchGuard sites for each conditional/assert.
(3) Create CallResult sites for each function call.
(4) Create OutBoundary sites for each return statement.
(5) Create ErrorSite sites for each partial operation (division, indexing, attribute access on `Optional`).
(6) Infer morphisms from control-flow and data-flow analysis.
(7) Identify overlaps via SSA-level reaching definitions.

4.2 Local Solving

At each site s, a *theory solver* computes the local section $\sigma_s \in \mathcal{S}em(s)$ by dispatching to the appropriate decision procedure:

- *Linear arithmetic*: for comparison guards ($x > 0$, $len(a) \geq k$).
- *Boolean*: for conjunctions/disjunctions of guards.
- *Tensor shapes*: for NumPy/PyTorch dimension constraints.
- *Nullability*: for `None` checks.
- *Collection*: for list/dict invariants.

Algorithm 1 Backward safe-input synthesis

Require: Cover $\mathcal{U}$, solved sections $\{\sigma_s\}$
Ensure: Input boundary sections $\{\rho_s \mid s \in \partial_{\text{in}}\}$

```
W ← error sites with viability predicates
while W ≠ ∅ do
    (s, σ) ← W.pop()
    if s ∈ ∂_in then
        Record ρ_s ← σ
    else
        for all morphisms f : t → s do
            σ′ ← Sem(f)^†(σ)                    ▷ pullback (adjoint of restriction)
            if t is φ-node then merge by ⊓
            end if
            W.push(t, σ′)
        end for
    end if
end while
```

Algorithm 2 Forward output synthesis

Require: Cover $\mathcal{U}$, input sections $\{\rho_s\}$
Ensure: Output boundary sections $\{\rho_s \mid s \in \partial_{\text{out}}\}$

```
W ← input boundary sites with ρ_s
while W ≠ ∅ do
    (s, σ) ← W.pop()
    if s ∈ ∂_out then
        Record ρ_s ← σ
    end if
    for all morphisms f : s → t do
        σ′ ← Sem(f)(σ)                          ▷ restriction
        if t is φ-node with multiple incoming then
            Accumulate; merge by ⊔ when all arrive
        else
            W.push(t, σ′)
        end if
    end for
end while
```

Each theory solver is *sound*: the computed section overapproximates the concrete states reachable at that site.

4.3 Bidirectional Fixed-Point Synthesis

The bidirectional synthesis alternates backward (Algorithm **??**) and forward (Algorithm **??**) passes until convergence:

- **Backward** (error → input): propagate viability requirements via pullback along morphisms. Produces *safe input requirements* ρ_{in}.

- **Forward** (input → output): push refined sections via restriction. Produces *maximal output guarantees* ρ_{out}.
- **Convergence check**: if no new sections are produced and the obstruction set stabilizes, stop. If $\check{H}^1 = 0$ (gluing succeeds), stop early with a global section.

4.4 Interprocedural Section Transport

For a call $y = g(x)$ in function f:

(1) **Actual → Formal**: restrict f's section at the call site to g's argument boundary via the actual-to-formal morphism.
(2) **Analyze** g: produce g's function summary (boundary sections).
(3) **Return → Caller**: transport g's output boundary back to f's call-result site.

This is an instance of the *transitivity axiom*: the cover of f is refined by the covers of its callees.

5 Three Applications

5.1 Bug Finding via $\check{H}^1$

Theorem 5.1 (Bugs as gluing obstructions). *A program P has a bug of class $\mathcal{B}$ if and only if $\check{H}^1(\mathcal{U}_P, \mathcal{S}em_{\mathcal{B}}) \neq 0$, where $\mathcal{S}em_{\mathcal{B}}$ is the presheaf encoding the safety property for bug class $\mathcal{B}$.*

Each bug class defines a presheaf by specifying viability predicates at error sites: division-by-zero requires divisor $\neq 0$, index out-of-bounds requires $0 \leq i < len(a)$, null dereference requires receiver $\neq$ `None`, etc.

Proposition 2 (rk $\check{H}^1$ = minimum fixes). *Over $\mathbb{F}_2$, rk $\check{H}^1(\mathcal{U}_P, \mathcal{S}em)$ equals the minimum number of independent code changes needed to resolve all obstructions.*

Proof. Build the coboundary matrix $\partial_0 : C^0(\mathbb{F}_2) \to C^1(\mathbb{F}_2)$ with rows indexed by overlaps and columns by sites: $(\partial_0)_{(i,j),k} = 1$ iff $k \in \{i, j\}$. Then rk $\check{H}^1 = \dim\ker\delta^1 - \dim\operatorname{im}\delta^0$. Each independent generator of $\check{H}^1$ is an obstruction that cannot be removed by adjusting sections at a single site (it is not a coboundary). Therefore, resolving all obstructions requires at least rk $\check{H}^1$ independent modifications. Conversely, modifying rk $\check{H}^1$ sites suffices: by linear algebra over $\mathbb{F}_2$, the space of coboundaries has codimension rk $\check{H}^1$ in the kernel, so rk $\check{H}^1$ generators span all non-trivial obstructions. □

5.2 Equivalence via Descent

Theorem 5.2 (Descent for equivalence). *Let f, g be two programs with semantic presheaves $\mathcal{S}em_f, \mathcal{S}em_g$ over a common cover $\mathcal{U}$. Then f and g are equivalent (there exists a natural isomorphism $\eta : \mathcal{S}em_f \xrightarrow{\sim} \mathcal{S}em_g$) if and only if $\check{H}^1(\mathcal{U}, \mathcal{I}so(\mathcal{S}em_f, \mathcal{S}em_g)) = 0$.*

Proof sketch. (⇐): The local isomorphisms $\{\eta_i : \mathcal{S}em_f(u_i) \xrightarrow{\sim} \mathcal{S}em_g(u_i)\}$ define transition functions $g_{ij} = \eta_j|_{u_{ij}} \circ \eta_i^{-1}|_{u_{ij}}$. $\check{H}^1 = 0$ means all g_{ij} satisfy the cocycle condition and are coboundaries, so they can be "straightened" to identities. The adjusted local isomorphisms agree on overlaps and glue (by the sheaf condition for $\mathcal{I}so$) to a global isomorphism.

(⇒): A global isomorphism η restricts to local isomorphisms with trivial transition functions ($g_{ij} = \mathrm{id}$), so the 1-cocycle is zero.

Full proof mechanized in Lean: `DeppyProofs.Descent.descent_theorem`. □

5.3 Specification Verification via Product Covers

Given a specification with k paths and m conjuncts, the *product cover* $\mathcal{U} = \{(p_i, c_j)\}_{i \leq k, j \leq m}$ creates one site per path-conjunct pair. Each verification condition (VC) is small: one path, one conjunct.

Proposition 3 (VC reduction). *Let n_{overlap} be the number of non-trivial overlaps in the product cover. Then $\check{H}^1$-based elimination reduces the number of VCs from $O(km)$ to $O(n_{\text{overlap}})$. Proof transfer across overlaps further reduces the count: if (p_1, c_j) is proved and $p_1 \cap p_2 \neq \emptyset$, then (p_2, c_j) is discharged.*

6 Metatheory

6.1 Soundness

We define the concrete semantics $[\![P]\!]$ as the collecting semantics: for each site s, the set of concrete states reachable at s during any execution. The abstraction function $\alpha : \wp(\text{State}) \to \mathcal{S}em(s)$ maps a set of states to the tightest section describing them. The concretization $\gamma : \mathcal{S}em(s) \to \wp(\text{State})$ maps a section to the states it admits. The pair (α, γ) forms a Galois connection: $\alpha(S) \sqsubseteq \sigma \iff S \subseteq \gamma(\sigma)$.

Theorem 6.1 (Soundness). *If $\check{H}^1(\mathcal{U}_P, \mathcal{S}em) = 0$, then for every error site e with viability predicate ψ_e, every concrete state reachable at e satisfies ψ_e. That is: the program has no bugs of the analyzed class.*

Proof. Suppose for contradiction that some concrete state c reaches error site e and violates ψ_e. Since the analysis is sound (each local section overapproximates the reachable states), $c \in \gamma(\sigma_e)$. But σ_e was computed to satisfy the viability predicate: $\gamma(\sigma_e) \subseteq \psi_e$.

Now, c reached e via a path through sites $s_1, \ldots, s_k = e$. At each site, the section overapproximates c. At the overlap between s_{k-1} and e, the section at s_{k-1} includes c (which violates ψ_e) while the viability predicate at e excludes it. Therefore, the restrictions of $\sigma_{s_{k-1}}$ and σ_e to the overlap disagree: there is a nonzero 1-cocycle. Since this cocycle cannot be resolved by local adjustment (the concrete state c witnesses the genuine disagreement), it represents a nonzero class in $\check{H}^1$, contradicting $\check{H}^1 = 0$.

Formalized in Lean: `DeppyProofs.Soundness.soundness`. □

6.2 Completeness Relative to the Cover

Theorem 6.2 (Relative completeness). *If a concrete bug exists (a reachable state at an error site violates the viability predicate) and the cover $\mathcal{U}$ is* fine enough *(every concrete execution path passes through at least one cover site), and the refinement lattice $\mathcal{R}$ is* expressive enough *(every viability predicate violation is expressible as a lattice element), then $\check{H}^1 \neq 0$.*

Proof. By the bug hypothesis, there exists a state c at error site e violating ψ_e. By "fine enough," c is abstracted at some site s adjacent to e. By "expressive enough," the section at s distinguishes c from safe states. Therefore, the overlap (s, e) has a genuine disagreement (nonzero 1-cocycle) that is not a coboundary. □

6.3 Mayer-Vietoris

Theorem 6.3 (Mayer-Vietoris for programs). *Decompose the cover $\mathcal{U}$ at a branch into sub-covers $\mathcal{A}$ and $\mathcal{B}$ with overlap $\mathcal{A} \cap \mathcal{B}$. There is an exact sequence:*

$$0 \to \check{H}^0(\mathcal{U}) \to \check{H}^0(\mathcal{A}) \oplus \check{H}^0(\mathcal{B}) \to \check{H}^0(\mathcal{A} \cap \mathcal{B}) \xrightarrow{\delta} \check{H}^1(\mathcal{U}) \to \check{H}^1(\mathcal{A}) \oplus \check{H}^1(\mathcal{B}) \to \check{H}^1(\mathcal{A} \cap \mathcal{B})$$

When the cover has no non-trivial triple overlaps ($\check{H}^2(\mathcal{U}) = 0$, which holds when each pair of sites has at most one overlap), the rank formula simplifies to:

$$\text{rk } \check{H}^1(\mathcal{U}) = \text{rk } \check{H}^1(\mathcal{A}) + \text{rk } \check{H}^1(\mathcal{B}) - \text{rk } \check{H}^1(\mathcal{A} \cap \mathcal{B}) + \text{rk im}(\delta)$$

Proof sketch. The semantic presheaf $\mathcal{S}em$ takes values in $\mathbb{F}_2$-vector spaces (each overlap is "agree" or "disagree"). The sub-covers $\mathcal{A}, \mathcal{B}$ form an excisive pair (their union is $\mathcal{U}$ and their intersection is a sub-cover of both). The Mayer-Vietoris sequence is a standard consequence of the

short exact sequence of Čech complexes: $0 \to C^\bullet(\mathcal{U}) \to C^\bullet(\mathcal{A}) \oplus C^\bullet(\mathcal{B}) \to C^\bullet(\mathcal{A} \cap \mathcal{B}) \to 0$. The connecting homomorphism δ maps an element of $\check{H}^0(\mathcal{A} \cap \mathcal{B})$ (a compatible family on the overlap that extends to both branches independently but not globally) to the obstruction in $\check{H}^1(\mathcal{U})$. □

Engineering consequence. Modify branch $\mathcal{A}$: recompute $\check{H}^1(\mathcal{A})$ and $\check{H}^0(\mathcal{A} \cap \mathcal{B})$; the global $\check{H}^1$ follows algebraically. This is the formal basis for Deppy's incremental analysis.

6.4 Fixed-Point Convergence

Theorem 6.4 (Convergence). *If the refinement lattice $\mathcal{R}$ has finite height h and the cover $\mathcal{U}$ has n sites, then the bidirectional synthesis (Algorithms ??–??) converges in at most $O(n \cdot h)$ iterations.*

Proof. Each iteration of the backward pass strictly increases the information content (in the $\sqsubseteq$ order) of at least one section, or leaves all sections unchanged (convergence). Since $\mathcal{R}$ has height h, each section can be strengthened at most h times. With n sites, the total number of strengthening steps is at most $n \cdot h$. Each forward pass is similarly bounded. Alternating backward and forward passes therefore converges in $O(n \cdot h)$ total iterations.

In practice, the refinement lattice is the lattice of predicates in linear arithmetic over a finite set of program variables. Quantifier elimination ensures the predicate domain is finite, giving a concrete bound on h. □

7 Evaluation

We evaluate Deppy on five research questions:

RQ1 Does $\check{H}^1$-based analysis detect real bugs?
RQ2 Does rk $\check{H}^1$ correctly predict minimum fix sets?
RQ3 Does the descent criterion correctly classify equivalences?
RQ4 Does the product-cover reduce specification VCs?
RQ5 Does Mayer-Vietoris enable incremental analysis?

7.1 Setup

Deppy is implemented in Python with the Čech pipeline as its core. The Z3 solver [?] is used for local solving and overlap agreement checking. Experiments run on a MacBook Pro (Apple M-series, 16 GB RAM). Key metatheorems are mechanized in 1 259 lines of Lean 4 with Mathlib.

The benchmark suite comprises 375 programs: 133 bug-detection programs spanning 22 bug classes (division by zero, index out of bounds, key error, null dereference, SQL injection, race conditions, type confusion, infinite loops, assertion failures, and more), 134 equivalence pairs (including torch/triton/numpy patterns, recursion–iteration equivalence, and mutation-vs-copy semantics), and 108 specification checks (from simple postconditions to multi-path invariant verification).

7.2 RQ1: Bug Detection

Task	Precision	Recall	F1
Bug detection (133 programs)	69%	**100%**	81%

Deppy achieves **100% recall** with zero false negatives: every bug in the suite is detected. Precision is 69% (F1 = 81%); false positives arise on safe programs with complex guard patterns (loop-bounded indexing, `defaultdict`, numerical stability tricks). On the same benchmarks (unannotated Python), mypy and pyright report *zero* findings because they require type annotations.

```python
# Deadlock: lock-order inversion (detected)
import threading
lock_a, lock_b = threading.Lock(), threading.Lock()

def task1():
    with lock_a:         # acquires A
        with lock_b:     # then B
            return 1

def task2():
    with lock_b:         # acquires B
        with lock_a:     # then A ← deadlock
            return 2
```

The site category has two synchronization sites, one per function. Each acquires a lock set: $L_1 = \{A, B\}$ and $L_2 = \{B, A\}$. The intersection $L_1 \cap L_2 = \{A, B\}$ is non-empty with reversed acquisition order. The presheaf section at task1's inner `with` requires *lock B available*, but task2's section at the same point holds lock B and waits for A. The sections disagree on the synchronization overlap: a non-trivial 1-cocycle in the lock-order presheaf.

```python
# Data race: unsynchronized shared mutation (detected)
import threading, ctypes
shared = ctypes.c_int32(0)

def add(n):
    for _ in range(n):
        shared.value += 1   # ← RACE_CONDITION

def run():
    threads = [threading.Thread(target=add,
              args=(10**8,)) for _ in range(4)]
    for t in threads: t.start()
    for t in threads: t.join()
    return shared.value    # ← INTEGER_OVERFLOW
```

Two independent obstructions: (1) `shared.value += 1` is a read-modify-write on a shared `c_int32` without a lock, producing a race-condition cocycle in the concurrency presheaf; (2) `ctypes.c_int32` has a bounded section ($[-2^{31}, 2^{31})$), and 4×10^8 exceeds it, producing an overflow obstruction. rk $\check{H}^1 = 2$: two independent fixes needed (add a lock, widen the type).

7.3 RQ2: $\check{H}^1$ Localization

The rank rk $\check{H}^1$ correctly predicts the minimum number of independent fixes in 102/133 programs (77%). Overestimation occurs when the analysis generates spurious obstructions on complex safe programs (loop-guarded indexing); these inflate rk $\check{H}^1$.

7.4 RQ3: Equivalence

	Correct	Total
Equivalent (true positive)	69	70
Non-equivalent (true negative)	64	64
Overall	133	134 (99%)

Soundness guarantee: zero false equivalences across all 134 pairs. Every non-equivalent pair is correctly identified, including torch-specific patterns (L1 vs. L2 norm) and mutation-vs-copy semantics.

```
# Two implementations of binary search: correctly EQUIVALENT
def search_a(xs, target):
    lo, hi = 0, len(xs) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if xs[mid] == target: return mid
        elif xs[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1

def search_b(xs, target):
    lo, hi = 0, len(xs)     # ← half-open interval
    while lo < hi:          # ← strict inequality
        mid = lo + (hi - lo) // 2
        if xs[mid] < target: lo = mid + 1
        elif xs[mid] > target: hi = mid
        else: return mid
    return -1
```

These use different loop invariants (closed vs. half-open interval), different midpoint formulas ($\lfloor (l+h)/2 \rfloor$ vs. $l + \lfloor (h-l)/2 \rfloor$), and different branch structures. Runtime sampling on 64 input pairs verifies identical return values. The isomorphism presheaf $\mathcal{I}so(\mathcal{S}em_f, \mathcal{S}em_g)$ has $\check{H}^1 = 0$.

```
# Fibonacci off-by-one: correctly INEQUIVALENT
def fib_a(n):              def fib_b(n):
    if n <= 1: return n        if n <= 1: return n
    a, b = 0, 1                a, b = 0, 1
    for _ in range(2, n+1):    for _ in range(2, n):
        a, b = b, a + b            a, b = b, a + b
    return b                   return b
    # fib(5) = 5              # fib(5) = 3
```

The loop bound differs by one: `range(2, n+1)` vs. `range(2, n)`. Z3 encodes both return expressions as functions of `n` and finds a counterexample at $n = 5$: $f(5) = 5 \neq 3 = g(5)$. The descent criterion correctly reports $\check{H}^1 \neq 0$.

7.5 RQ4: Specification Verification

	Correct	Total
Spec satisfaction	106	108 (98%)

Soundness guarantee: zero false satisfactions. No violated specification is approved. The two errors are conservative rejections (correct programs marked unsatisfied due to incomplete guard resolution).

```
# Broken bubble sort: correctly REJECTED against spec
def broken_bubble_sort(arr):
    a = list(arr)
    n = len(a)
    for i in range(n):
        for j in range(n - i - 2): # bug: should be n-i-1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
    return a

# Spec: result == sorted(arr)
```

The off-by-one in the inner loop bound (`n - i - 2` instead of `n - i - 1`) means the last adjacent pair in each pass is never compared. Static analysis finds no structural bugs (no division, no null dereference), but runtime sampling discovers counterexample $arr = [3, 1, 2] \mapsto [1, 3, 2] \neq [1, 2, 3]$. The product-cover VC for the path $\langle j = n{-}i{-}2 \rangle \times \langle result = \texttt{sorted}(arr) \rangle$ is not discharged.

```
# Merge sort: correctly VERIFIED against spec
def merge_sort(arr):
    if len(arr) <= 1:
        return list(arr)
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    merged, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i]); i += 1
        else:
            merged.append(right[j]); j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged

# Spec: result == sorted(arr)
```

The product cover factors the specification into path-conjunct VCs. The recursive structure creates a cover hierarchy via the Grothendieck transitivity axiom: the cover of `merge_sort(arr)` refines into covers for the two recursive calls and the merge loop. All VCs are discharged: static analysis finds no obstructions ($\check{H}^1 = 0$), and runtime sampling confirms $result = \texttt{sorted}(arr)$ on all 200 sampled inputs.

7.6 RQ5: Incremental Analysis

On a 3-function module, the Mayer-Vietoris incremental analysis achieves a 1.3× speedup over full re-analysis when modifying one function. The speedup is modest because the benchmark programs are small; we expect the benefit to grow with codebase size as the Mayer-Vietoris exact sequence eliminates recomputation of unchanged sub-covers.

7.7 Separation Results

We identify three problems where the sheaf-cohomological approach is provably more capable than traditional program analysis.

Theorem 7.1 (Polynomial minimum-fix count). *Given a program with n sites and m overlaps, the minimum number of independent code changes needed to eliminate all bugs is computed in $O(m^2 n)$ time via Gaussian elimination on the coboundary matrix $\partial_0 : C^0(\mathbb{F}_2) \to C^1(\mathbb{F}_2)$.*

In contrast, computing the minimum fix count from the output of a traditional abstract interpretation (a set of error sites with no overlap structure) is NP-hard: it reduces to Minimum Hitting Set on the family of error-contributing site subsets.

Proof. The coboundary matrix ∂_0 has m rows (overlaps) and n columns (sites). $\operatorname{rk} \check{H}^1 = \dim \ker \delta^1 - \operatorname{rk} \partial_0$, computable by Gaussian elimination in $O(m^2 n)$. This directly equals the minimum number of independent fixes (Proposition **??**).

For traditional analysis: the output is a set E of error sites. Each possible fix modifies one site and eliminates the errors at that site. Finding the minimum number of sites to modify so that every error in E is covered is Minimum Hitting Set, which is NP-hard even to approximate within factor $(1-\epsilon)\ln|E|$ *(Dinur & Steurer, 2014)*. The sheaf approach avoids this because the coboundary matrix algebraically encodes the *dependencies* between errors—two errors that share a coboundary (are resolvable by the same local adjustment) are identified by the quotient $\ker \delta^1 / \operatorname{im} \delta^0$, collapsing the combinatorial search to a linear-algebra computation. □

Theorem 7.2 (Exact incremental update). *Let $\mathcal{U} = \mathcal{A} \cup \mathcal{B}$ be a cover decomposition at a branch point. If only sub-cover $\mathcal{A}$ is modified, the updated global cohomology is computed by:*

$$\check{H}^1(\mathcal{U}') = \check{H}^1(\mathcal{A}') \oplus \check{H}^1(\mathcal{B}) \oplus \operatorname{im}(\delta') \;/\; \check{H}^1(\mathcal{A}' \cap \mathcal{B})$$

*requiring only recomputation of $\check{H}^1(\mathcal{A}')$ and the connecting homomorphism. This is **exact**: no precision is lost.*

Abstract interpretation's incremental re-analysis, by contrast, must re-solve the global fixpoint (since widening at join points destroys the algebraic structure *needed for local update), incurring $O(n \cdot h)$ cost for the full program even when only $O(|\mathcal{A}|)$ sites change.*

Proof. Exactness follows directly from the Mayer-Vietoris exact sequence (Theorem **??**): the sequence is natural in the sub-covers, so replacing $\mathcal{A}$ with $\mathcal{A}'$ yields a new exact sequence from which $\check{H}^1(\mathcal{U}')$ is determined. Sub-cover $\mathcal{B}$ is unchanged, so $\check{H}^1(\mathcal{B})$ is reused. Only $\check{H}^1(\mathcal{A}')$, $\check{H}^0(\mathcal{A}' \cap \mathcal{B})$, and the connecting homomorphism δ' need recomputation.

Capability	Deppy	AI	Ref. Types	Hoare
Error localization	rk $\check{H}^1$	—	—	1 VC
Compositionality	MV (exact)	Widen (lossy)	Modules	Frame
Equiv. checking	Descent (complete)	—	—	Rel.
Incremental	Algebraic (exact)	Re-fix ($O(nh)$)	Re-check	Re-verify
Fix count	$O(m^2 n)$	NP-hard	—	—

Table 1. Separation results vs. abstract interpretation (AI), refinement types, and Hoare logic. MV = Mayer-Vietoris. Three entries are strict separations (Theorems ??–??).

For abstract interpretation: the fixpoint equation at a join point is $x = f(x) \sqcup g(x)$ where f depends on $\mathcal{A}$ and g on $\mathcal{B}$. Widening replaces $\sqcup$ with ∇, which is non-algebraic (non-associative, non-commutative in general). When f changes, the widened fixpoint must be recomputed globally because ∇ does not distribute over the decomposition. □

Theorem 7.3 (Complete equivalence criterion). *The descent criterion $\check{H}^1(\mathcal{U}, \mathcal{I}so) = 0$ is both **sound** and **complete** for equivalence relative to the cover: $f \equiv_{\mathcal{U}} g$ if and only if $\check{H}^1 = 0$.*

No finite abstract domain D yields a complete equivalence criterion: for any D, there exist equivalent programs $f \equiv g$ such that $\alpha_D(f) \neq \alpha_D(g)$ (the abstractions differ even though the concrete semantics agree).

Proof. Soundness and completeness of descent were proved in Theorem ??.

For the incompleteness of abstract domains: let D be any finite abstract domain and $\gamma : D \to \wp(\text{State})$ its concretization. Since D is finite, γ partitions states into finitely many equivalence classes. For any such partition, there exist programs f, g with $[\![f]\!] = [\![g]\!]$ (identical denotational semantics) but $\alpha(f) \neq \alpha(g)$: choose f that exercises the abstract domain's precision limits differently from g (e.g., f uses a loop, g uses recursion; both compute the same function, but the widening trajectories of f and g in D converge to different fixpoints).

The sheaf approach avoids this because the isomorphism presheaf $\mathcal{I}so$ checks *local* equivalence (at each site) and the descent theorem glues local equivalences into a global one. The cover, not the abstract domain, determines the precision. □

7.8 Comparison Summary

Table ?? summarizes these separation results.

8 Related Work

Abstract interpretation. Cousot and Cousot [?] established local-to-global reasoning via lattice fixpoints. Our presheaf framework subsumes this: local sections correspond to abstract domain elements, restriction maps to transfer functions, and the gluing condition to the fixpoint condition. The novelty is the *cohomological structure*: $\check{H}^1$ provides obstruction counting, localization, and compositionality (Mayer-Vietoris) that flat abstract interpretation lacks.

Refinement types. Liquid Haskell [?] and Flux [?] synthesize refinement types via SMT. Deppy shares the refinement-type substrate but adds the sheaf structure: sections are organized into covers with overlaps, enabling $\check{H}^1$-based error localization. Liquid Haskell has no analogue of "minimum independent fixes."

Sheaves in other domains. Goguen [?] proposed sheaf semantics for concurrent programs, modeling information flow as a sheaf over a poset of security levels. This is a *semantic* use of sheaves; our work uses sheaves for *analysis algorithms*. Ghrist and colleagues [? ?] use cellular sheaves for sensor networks and opinion dynamics. The cohomological obstruction theory is closest to our work, but in a different domain (data fusion, not programs). Abramsky and Brandenburger [?] use sheaves for quantum contextuality, connecting non-locality to cohomological obstructions. Again, the mathematical machinery is similar but the domain is different.

Interprocedural analysis. IFDS/IDE [?] computes interprocedural summaries via graph reachability. Our Grothendieck transitivity axiom (§??) is the categorical generalization: f's cover is refined by callee covers, and summaries are section transport along call-boundary morphisms.

Separation logic. The frame rule of separation logic [?] enables local reasoning by asserting that disjoint heap regions are independent. Our restriction maps along site morphisms serve a similar purpose, but for refinement-type information rather than heap ownership. The sheaf axioms (stability, transitivity) generalize the frame rule to non-heap properties.

9 Conclusion

We have shown that program analysis can be organized as Čech cohomology of a semantic presheaf over the program's site category. This yields three results unavailable in prior work: rk $\check{H}^1$ counts minimum independent fixes, descent provides sound and complete equivalence checking, and Mayer-Vietoris enables compositional incremental analysis. The implementation, Deppy, demonstrates these capabilities on real Python programs.

Mechanized proofs. Soundness, the sheaf condition equivalence, $\check{H}^0$ characterization, the descent theorem, Mayer-Vietoris localization, incremental soundness, and fixed-point convergence are formalized in Lean 4 with Mathlib (1 259 lines across 7 modules).

Limitations. Completeness is relative to the cover and predicate domain. Bug-detection precision (69%) is limited by false positives on safe programs with complex guard patterns (defaultdict, loop-bounded indexing, numerical stability tricks). The H^1 rank prediction (77%) is limited by spurious obstructions inflating the rank. The framework currently targets single-threaded Python; full concurrency support (beyond race-condition detection) would require extending the site category with synchronization morphisms.

References

[1] P. Cousot and R. Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *POPL*, 1977.

[2] R. Hindley. The principal type-scheme of an object in combinatory logic. *Transactions of the AMS*, 146:29–60, 1969.

[3] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10), 1969.

[4] E. Clarke, E. Emerson, and A. Sistla. Automatic verification of finite-state concurrent systems using temporal logic specifications. *ACM TOPLAS*, 8(2), 1986.

[5] T. Reps, S. Horwitz, and M. Sagiv. Precise interprocedural dataflow analysis via graph reachability. In *POPL*, 1995.

[6] N. Vazou, E. Seidel, R. Jhala, D. Vytiniotis, and S. Peyton Jones. Refinement types for Haskell. In *ICFP*, 2014.

[7] N. Lehmann, A. T. Geller, N. Vazou, and R. Jhala. Flux: Liquid types for Rust. In *PLDI*, 2023.

[8] J. A. Goguen. Sheaf semantics for concurrent interacting objects. *Mathematical Structures in Computer Science*, 2(2), 1992.

[9] R. Ghrist. Elementary Applied Topology. Createspace, 2014.
[10] J. Hansen and R. Ghrist. Opinion dynamics on discourse sheaves. *SIAM Journal on Applied Mathematics*, 81(5), 2021.
[11] S. Abramsky and A. Brandenburger. The sheaf-theoretic structure of non-locality and contextuality. *New Journal of Physics*, 13(11), 2011.
[12] P. O'Hearn, J. Reynolds, and H. Yang. Local reasoning about programs that alter data structures. In *CSL*, 2001.
[13] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *TACAS*, 2008.
[14] R. Widyasari, S. Q. Sim, C. Lok, H. Qi, J. Phan, Q. Tay, C. Tan, F. Wee, J. E. Tan, Y. Yieh, B. Goh, F. Thung, H. J. Kang, T. Hoang, D. Lo, and E. L. Ouh. BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies. In *ESEC/FSE*, 2020.

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.